

The DHS hiring surge in ICE & CBP: how big, did it stick, and who is leaving?

The question. In 2025 the Department of Homeland Security launched a large hiring push, concentrated in ICE and CBP, and the numbers were publicly disputed. Using federal personnel records we ask three things:

1. **How big was the surge**, compared with normal ICE and CBP hiring?
2. **Did it stick** — did the new hires actually show up on the payroll, and are they *staying*?
3. **Who is leaving** — are the people leaving the *surge hires themselves*, are they **quitting or being let go**, and is the departure rate **higher than for similar hires**?

The data. OPM/EHRI personnel records, published monthly and mirrored at `impactproject/opm-ehri-data`. Three files: **hires** (OPM calls them *accessions*), **departures** (*separations*), and a monthly **headcount** snapshot. Each row is a *count of people* sharing a set of attributes (job, grade, pay plan, etc.) — **not** an individual record, so we can't literally follow one person from hire to exit. We work around that by identifying the surge hires through the **jobs they were hired into** (§3).

Method, checks, and definitions are in the README; §0 below runs the checks live against the data.

```
In [1]: import sys; sys.path.insert(0, "src")
import surge
import pandas as pd, matplotlib.pyplot as plt, matplotlib.ticker as mtick, n
from IPython.display import display, Markdown
pd.set_option("display.max_columns", 30)

# ---- tables: thousands separators, whole numbers (no trailing ".0"), % whe
def show(df, pct=(), index=False, na_rep="--"):
    """Render a DataFrame as a tidy HTML table. Numeric columns get thousand
    columns that are whole numbers lose the trailing '.0'; columns named in
    fmt = {}
    for c in df.columns:
        col = df[c]
        if c in pct:
            fmt[c] = lambda v: "" if pd.isna(v) else f"{v:,.1f}%"
        elif pd.api.types.is_numeric_dtype(col):
            s = col.dropna()
            whole = len(s) > 0 and bool((s == s.round()).all())
            fmt[c] = (lambda v: "" if pd.isna(v) else f"{int(round(v)):,}")
                        else (lambda v: "" if pd.isna(v) else f"{v:,.1f}")
    sty = df.style.format(fmt, na_rep=na_rep)
    return sty if index else sty.hide(axis="index")

def ym_label(ym): # "202512" -> "Dec 2025"
```

```

    return pd.to_datetime(str(ym), format="%Y%m").strftime("%b %Y")

# Occupational-series names, so tables show what a job code means (USAJOBS c
occ_names = pd.read_csv("data/occ_series_names.csv", dtype=str).set_index("c
def series_label(code): # "1801" -> "1801 - General Inspection, Investigati
    name = occ_names.get(str(code))
    return f"{code} - {name}" if name else str(code)

# ---- charts: a lighter, less cluttered theme + a small consistent palette
C_ICE, C_CBP, C_EXIT, C_MUTED, C_ALT = "#2166ac", "#e08214", "#c0392b", "#8a
SPAN = "#fdd9a0" # shading for the surge window
plt.rcParams.update({
    "figure.dpi": 120, "savefig.bbox": "tight", "font.size": 11,
    "axes.spines.top": False, "axes.spines.right": False, "axes.edgecolor":
    "axes.titlesize": 12, "axes.titleweight": "bold", "axes.titlepad": 10,
    "axes.labelcolor": "#444", "axes.labelsize": 10, "xtick.color": "#444",
    "axes.grid": True, "axes.axisbelow": True, "grid.color": "#dddddd", "gri
    "legend.frameon": False,
})
def comma_y(ax): # 30000 -> "30,000" on the y-axis
    ax.yaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _: f"{x:,.0f}")
def int_x(ax): # whole years on the x-axis (2010, not 2010.0)
    ax.xaxis.set_major_locator(mtick.MaxNLocator(integer=True))
    ax.xaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _: f"{int(x)}")
def date_x(ax): # clean, non-overlapping month/year labels on a date x-axis
    loc = mdates.AutoDateLocator() # default bounds pick sensible interval
    ax.xaxis.set_major_locator(loc)
    ax.xaxis.set_major_formatter(mdates.ConciseDateFormatter(loc))

con = surge.con()
def q(sql): return con.execute(sql).df()
ACC, SEP, EMP = surge.ACC, surge.SEP, surge.EMP
print("Cached extract:", ACC.split("/")[-1], "|", SEP.split("/")[-1], "|", E
print("Latest month in data:", ym_label(surge.LATEST_MONTH), "(the most rece

```

Cached extract: acc_ice_cbp.parquet | sep_ice_cbp.parquet | emp_ice_cbp.parquet

Latest month in data: Jul 2026 (the most recent 1–2 months are provisional)

0. Method checks

Every assumption this analysis leans on is checked against the data live, below, not just asserted — full rationale for each is in the README. Four checks: agency codes are stable and exclusive (①), summing across files by effective date doesn't double-count (②), the headcount decline is real and not a reporting artifact (③), and redaction preserves totals while the surge is genuine new hires, not transfers (④).

In [2]: *# CHECK 1 – agency identity & code stability. This audit (built offline by s*
the raw yearly files) lists EVERY DHS/DOJ office whose name mentions immig
confirms ICE=HSBB and CBP=HSBD are the only ICE/CBP codes, stable back to
(USCIS=HSAB, DOJ immigration courts=DJ12) are separate agencies our code t
NOTE: we match with regexp_matches(...), NOT `SIMILAR TO '%...%'` – in Duc
`%` is a literal, so that pattern silently matches nothing (a bug we hit a

```

audit = pd.read_csv("data/agency_code_audit.csv", dtype=str)
audit["accessions"] = pd.to_numeric(audit["accessions"], errors="coerce")
piv = (audit.pivot_table(index=["code", "agency_name"], columns="year",
                          values="accessions", aggfunc="first").reset_index()
piv.columns.name = None
display(show(piv, na_rep="")) # blank (not "-") for years an agency had no
print("Codes ever matching immigration/customs/border:", sorted(audit.code.u
print("=> ICE=HSBB and CBP=HSBD only; stable 2008-2026; HSAB (USCIS) and DJ1
print("The cached extract contains exactly:",
      "; ".join(f"{r.agy_code}={r.agency}" for _, r in q(f"SELECT DISTINCT ag

```

code	agency_name	2008	2012	2016	2018	2020	2022	2024	2025
DJ12	EXECUTIVE OFFICE FOR IMMIGRATION REVIEW	16	4	2	1	56	12	25	
HSAB	BUREAU OF CITIZENSHIP AND IMMIGRATION SERVICES	218							
HSAB	CITIZENSHIP AND IMMIGRATION SERVICES		164	92	162	7	229	949	1
HSBB	BUREAU OF IMMIGRATION AND CUSTOMS ENFORCEMENT	196							
HSBB	IMMIGRATION AND CUSTOMS ENFORCEMENT		53	26	73	85	65	216	192
HSBD	BUREAU OF CUSTOMS AND BORDER PROTECTION	723							
HSBD	CUSTOMS AND BORDER PROTECTION		243	128	324	448	329	472	516

Codes ever matching immigration/customs/border: ['DJ12', 'HSAB', 'HSBB', 'HSBD']

=> ICE=HSBB and CBP=HSBD only; stable 2008-2026; HSAB (USCIS) and DJ12 (DOJ courts) are excluded.

The cached extract contains exactly: HSBB=IMMIGRATION AND CUSTOMS ENFORCEMENT; HSBD=CUSTOMS AND BORDER PROTECTION

```

In [3]: # CHECK 2 – files add new actions, they don't re-state the whole history. For
# for the December-2025 surge month, show how that month's hires are split a
# them. If files re-stated history, one file would hold ~100% every time.
rows = []
for ym in ["202503", "202512"]:
    d = q(f"SELECT file_ym, SUM(count) n FROM '{ACC}' WHERE event_ym='{ym}'")
    for _, r in d.iterrows():
        rows.append({"Hires effective": ym_label(ym), "Reported in file": ym,
                    "Hires in that file": int(r.n),
                    "Share of the month": round(100*r.n/d.n.sum(), 1)})
display(show(pd.DataFrame(rows), pct=["Share of the month"]))
print("Mar 2025: ~98% lands in its own file (later files add small, DIFFEREN

```

```
print("Dec 2025: only ~30% lands in its own file; most December hires were p
print("=> Summing across files is REQUIRED; using one file per month would b
```

Hires effective	Reported in file	Hires in that file	Share of the month
Mar 2025	Mar 2025	498	97.5%
Mar 2025	Nov 2025	1	0.2%
Mar 2025	Feb 2026	5	1.0%
Mar 2025	Mar 2026	5	1.0%
Mar 2025	Jul 2026	2	0.4%
Dec 2025	Dec 2025	1,067	29.2%
Dec 2025	Jan 2026	1,952	53.3%
Dec 2025	Feb 2026	176	4.8%
Dec 2025	Mar 2026	233	6.4%
Dec 2025	Apr 2026	138	3.8%
Dec 2025	May 2026	25	0.7%
Dec 2025	Jun 2026	26	0.7%
Dec 2025	Jul 2026	43	1.2%

Mar 2025: ~98% lands in its own file (later files add small, DIFFERENT stragglers -> no double-count).

Dec 2025: only ~30% lands in its own file; most December hires were processed in January.

=> Summing across files is REQUIRED; using one file per month would badly undercount the surge.

```
In [4]: # CHECK 3 – how complete is a month after k months? (measured on settled 202
mat = q(f"""
    WITH x AS (SELECT event_ym, SUM(count) n,
        (CAST(substr(file_ym,1,4) AS INT)*12+CAST(substr(file_ym,5,2) AS INT))
        -(CAST(substr(event_ym,1,4) AS INT)*12+CAST(substr(event_ym,5,2) AS INT))
    FROM '{SEP}' WHERE event_ym BETWEEN '202401' AND '202411' GROUP BY 1, file_ym
    SELECT lag AS "Months after the month", ROUND(100.0*SUM(SUM(n)) OVER (ORDER BY lag))
    FROM x WHERE lag>=0 GROUP BY lag ORDER BY lag LIMIT 4
    """)
display(show(mat, pct=["% of final total present"]))
chk = q(f"""SELECT COUNT(DISTINCT snapshot_ym) snapshots FROM '{EMP}'""").sr
print(f"Headcount file: {chk} distinct snapshot months, one per file (verified)")
print("=> Normal months are ~99% complete right away; the recent SURGE month is not")
print("    anchor the 'headcount is declining' claim on the headcount snapshots")
```

Months after the month	% of final total present
0	98.8%
1	99.1%
2	99.1%
3	99.3%

Headcount file: 55 distinct snapshot months, one per file (verified) → no double-count of headcount.

=> Normal months are ~99% complete right away; the recent SURGE months settle more slowly, so we

anchor the 'headcount is declining' claim on the headcount snapshots, not on the monthly flows.

```
In [5]: # CHECK 4 – redaction preserves totals; and the surge is new hires, not transfers
tot, red, nn = q(f"""SELECT SUM(count) t,
    SUM(CASE WHEN duty_state='REDACTED' THEN count END) r,
    SUM(CASE WHEN TRY_CAST(count AS INT) IS NULL THEN 1 ELSE 0 END) nn
FROM '{ACC}' WHERE event_ym BETWEEN '202509' AND '202601'""").iloc[0]
print(f"Surge-window ICE+CBP hires: {int(tot):,} people; {int(red):,} have their work location REDACTED (hidden value, still counted); non-numeric counts = {int(nn):,}")
tr = q(f"""SELECT accession_category, SUM(count) n FROM '{ACC}'
    WHERE agy_code='{surge.ICE}' AND event_ym BETWEEN '202509' AND '202601'""")
nh = int(tr[tr.accession_category.str.startswith('NEW HIRE')].n.sum())
xf = int(tr[tr.accession_category.str.startswith('TRANSFER')].n.sum())
print(f"Surge ICE hires: NEW HIRE = {nh:,,} vs TRANSFER IN = {xf:,,} ({100*nh/xf}% transfers).")
print("=> The surge is genuinely new appointments, not staff moved over from other agencies.")
```

Surge-window ICE+CBP hires: 12,888 people; 12,603 have their work location REDACTED (hidden value, still counted); non-numeric counts = 0.

Surge ICE hires: NEW HIRE = 10,133 vs TRANSFER IN = 44 (0.4% transfers).

=> The surge is genuinely new appointments, not staff moved over from other agencies.

With the method checked out, the findings follow.

1. How big was the surge?

New hires per calendar year, by agency — genuine new hires only (the "NEW HIRE" category excludes transfers in from other agencies; those are handled in §5). The current year is partial.

```
In [6]: by_year = q(f"""
    SELECT substr(event_ym,1,4) AS year,
    SUM(CASE WHEN agy_code='{surge.ICE}' AND {surge.NEW_HIRE} THEN count END) AS ice_new_hires,
    SUM(CASE WHEN agy_code='{surge.CBP}' AND {surge.NEW_HIRE} THEN count END) AS cbp_new_hires
FROM '{ACC}' WHERE year BETWEEN '2018' AND '2026' GROUP BY 1 ORDER BY 1
""")
show(by_year.rename(columns={"year": "Year", "ice_new_hires": "ICE new hires", "cbp_new_hires": "CBP new hires"}))
```

Out [6]:

Year	ICE new hires	CBP new hires
2018	893	3,387
2019	1,137	4,344
2020	1,293	3,934
2021	740	3,215
2022	715	3,511
2023	1,031	4,106
2024	1,334	4,195
2025	10,474	5,173
2026	1,568	5,119

2018	893	3,387
2019	1,137	4,344
2020	1,293	3,934
2021	740	3,215
2022	715	3,511
2023	1,031	4,106
2024	1,334	4,195
2025	10,474	5,173
2026	1,568	5,119

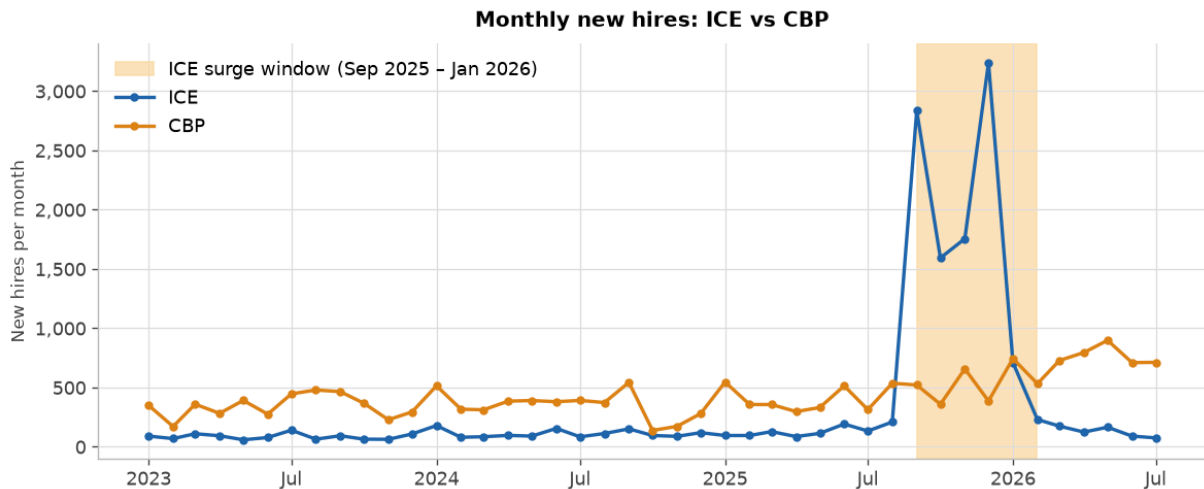
```
In [7]: pre_ice = by_year[(by_year.year >= "2018") & (by_year.year <= "2024")].ice_r
ice_2025 = int(by_year.loc[by_year.year == "2025", "ice_new_hires"].iloc[0])
cbp_2025 = by_year.loc[by_year.year == "2025", "cbp_new_hires"].iloc[0]
cbp_2024 = by_year.loc[by_year.year == "2024", "cbp_new_hires"].iloc[0]
display(Markdown(
    f"ICE normally hires roughly {int(pre_ice.min()):,}–{int(pre_ice.max()):,}:
    f"*In 2025 it hired {ice_2025:,} – about {ice_2025/pre_ice.max():.0f}×
    f"CBP's 2025 increase is small (about {cbp_2025/cbp_2024:.1f}× its 2024
    f"overwhelmingly an ICE event. The monthly view shows when it happ
    ))
```

ICE normally hires roughly 715–1,334 people a year. **In 2025 it hired 10,474 — about 8× normal.** CBP's 2025 increase is small (about 1.2× its 2024 level). So the surge is overwhelmingly an **ICE** event. The monthly view shows *when* it happened.

```
In [8]: monthly = q(f"""
    SELECT event_ym,
           SUM(CASE WHEN agy_code='{surge.ICE}' AND {surge.NEW_HIRE} THEN count END
           SUM(CASE WHEN agy_code='{surge.CBP}' AND {surge.NEW_HIRE} THEN count END
    FROM '{ACC}' WHERE event_ym >= '202301' GROUP BY 1 ORDER BY 1
    """)
monthly["date"] = pd.to_datetime(monthly.event_ym, format="%Y%m")
fig, ax = plt.subplots(figsize=(10, 4.2))
ax.axvspan(pd.Timestamp("2025-09-01"), pd.Timestamp("2026-01-31"), color=SPA
           label="ICE surge window (Sep 2025 – Jan 2026)")
ax.plot(monthly.date, monthly.ICE, marker="o", ms=4, lw=2, color=C_ICE, label=
ax.plot(monthly.date, monthly.CBP, marker="o", ms=4, lw=2, color=C_CBP, label=
ax.set(title="Monthly new hires: ICE vs CBP", ylabel="New hires per month")
comma_y(ax); date_x(ax); ax.legend(loc="upper left"); fig.tight_layout()

surge_win = monthly.query("'202509' <= event_ym <= '202601'")
surge_win_ice, surge_win_peak = int(surge_win.ICE.sum()), surge_win.loc[surge
print(f"ICE new hires, Sep 2025 – Jan 2026: {surge_win_ice:,}")
```

ICE new hires, Sep 2025 – Jan 2026: 10,133



```
In [9]: display(Markdown(
    f"The ICE surge is a sharp burst from Sep 2025 to Jan 2026 — {surge_
    f"months, peaking in {ym_label(surge_win_peak.event_ym)} — then it drops
    f"monthly levels. CBP shows no comparable spike; it ramps up gradually a
    ))
```

The ICE surge is a **sharp burst from Sep 2025 to Jan 2026** — 10,133 hires in five months, peaking in Dec 2025 — then it drops back to roughly normal monthly levels. CBP shows no comparable spike; it ramps up gradually and later, into 2026.

1b. How does this compare to *past* ICE hiring?

To see whether this is unusual for ICE, we extend the series back to 2006 (from the raw yearly files, pulled by `src/history.py`; the agency codes were verified stable back to 2008 in §0) and join it to the 2018-onward cached extract.

```
In [10]: hist = pd.read_csv("data/ice_cbp_hires_history.csv") # 2006–2017, from src/
long = (hist.pivot_table(index="year", columns="agy", values="new_hires", aggfunc="sum",
    .rename(columns={"HSBB":"ICE", "HSBD":"CBP"}))
recent = by_year.set_index("year").rename(columns={"ice_new_hires":"ICE", "cbp_new_hires":"CBP"})
recent.index = recent.index.astype(int)
arc = pd.concat([long, recent]).sort_index()
arc = arc[arc.index <= 2025] # 2026 is partial, drop it from the annual view
fig, ax = plt.subplots(figsize=(11, 4.4))
ax.plot(arc.index, arc.ICE, marker="o", ms=4, lw=2, color=C_ICE, label="ICE")
ax.plot(arc.index, arc.CBP, marker="o", ms=4, lw=2, color=C_CBP, label="CBP")
ax.annotate("CBP Border Patrol\nbuildup (2007–09)", (2008, arc.CBP.loc[2008]),
    textcoords="offset points", xytext=(6, 8), fontsize=9, color=C_CBP)
ax.annotate("2017 order to 'hire 10,000\nICE officers' -> no surge", (2018,
    textcoords="offset points", xytext=(-8, 30), fontsize=9, color="black",
    arrowprops=dict(arrowstyle="->", lw=.7, color="#888"))
ax.annotate(f"2025 ICE surge\n{int(arc.ICE.loc[2025]):,}", (2025, arc.ICE.loc[2025]),
    textcoords="offset points", xytext=(-72, -4), fontsize=10, color="red")
ax.set(title="New hires per year: ICE vs CBP, 2006–2025", ylabel="New hires per year",
    comma_y(ax); int_x(ax); ax.legend(loc="upper center"); fig.tight_layout()

ice_prior_peak, ice_prior_peak_yr = arc.ICE.loc[:2024].max(), int(arc.ICE.loc[:2024].max().index)
```

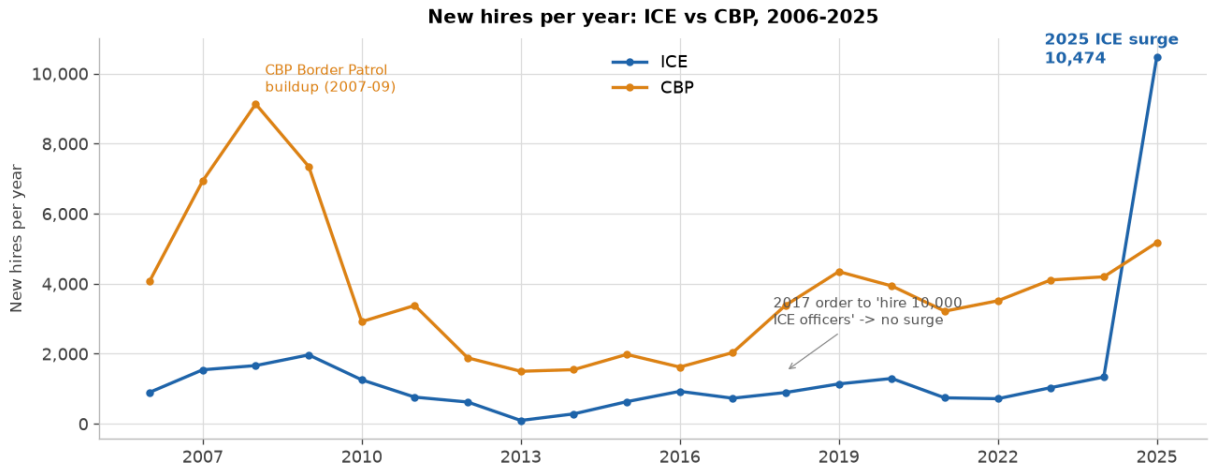
```

cbp_prior_peak, cbp_prior_peak_yr = arc.CBP.loc[:2024].max(), int(arc.CBP.loc[
post_order = arc.ICE.loc[2018:2020] # the three years after the 2017 "hire
print(f"ICE annual new hires – prior peak: {int(ice_prior_peak):,} in {ice_p
      f" | 2025: {int(arc.ICE.loc[2025]):,} ({arc.ICE.loc[2025]/ice_prior
print(f"CBP annual new hires – prior peak: {int(cbp_prior_peak):,} in {cbp_p

```

ICE annual new hires – prior peak: 1,966 in 2009 | 2025: 10,474 (5.3x the prior peak)

CBP annual new hires – prior peak: 9,130 in 2008



```

In [11]: cbp_buildup = arc.CBP.loc[2007:2009]
display(Markdown(
    f"*2025 is the biggest hiring year in ICE's record.* ICE's previous *a
    f"{int(ice_prior_peak):,} new hires ({ice_prior_peak_yr}); 2025's {int(a
    f"*{arc.ICE.loc[2025]/ice_prior_peak:.0f}x ICE's all-time record and {a
    f"its recent norm*. Notably, the **2017 executive order that directed t
    f"officers produced no visible surge** – ICE hiring stayed near {int(pos
    f"{int(post_order.max()):,} a year through 2020. The only thing in this
    f"is **CBP in 2007–2009**, when the Bush-era Border Patrol buildup drove
    f"{int(cbp_buildup.max()):,} hires a year. So the 2025 ICE surge resembl
    f"larger scale; and CBP's most recent rapid entry-officer push, in 2019–
    f"first-year attrition (§4b)."
))

```

2025 is the biggest hiring year in ICE's record. ICE's previous *annual* record was 1,966 new hires (2009); 2025's 10,474 is **5x ICE's all-time record and 8x its recent norm**.

Notably, the **2017 executive order that directed hiring 10,000 more ICE officers**

produced no visible surge — ICE hiring stayed near 893–1,293 a year through 2020. The only thing in this data that *looks* like 2025 is **CBP in 2007–2009**, when the Bush-era Border Patrol buildup drove 6,935–9,130 hires a year. So the 2025 ICE surge resembles CBP's 2007–2009 buildup, at a larger scale; and CBP's most recent rapid entry-officer push, in 2019–21, also saw unusually high first-year attrition (§4b).

2. Did it stick — did the hires stay on the rolls?

Hires are a monthly *flow*. The headcount file is the *level* — how many people are actually on the payroll each month. If the surge is real and the hires stayed, ICE headcount should jump and hold.

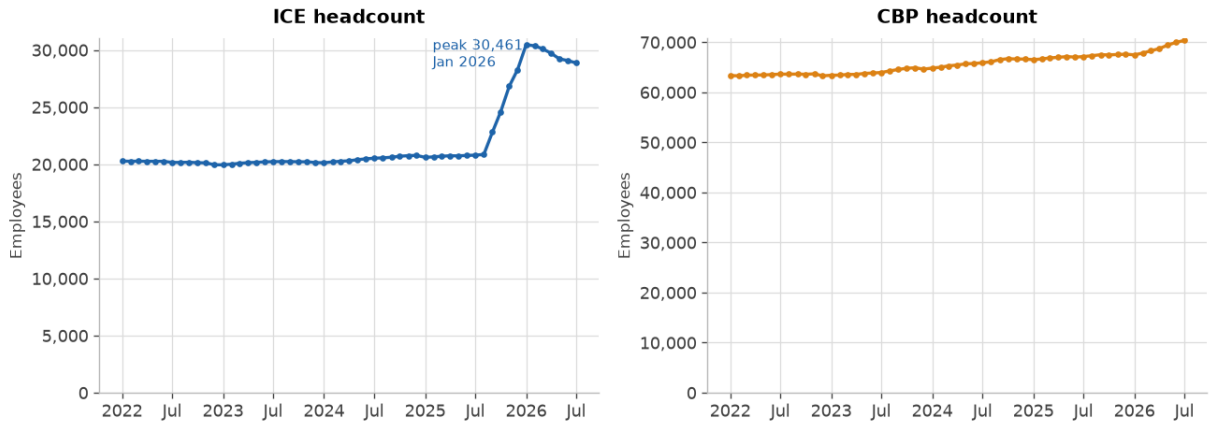
```

In [12]: head = q(f"""
SELECT snapshot_ym,
      SUM(CASE WHEN agy_code='{surge.ICE}' THEN count END) AS ICE,
      SUM(CASE WHEN agy_code='{surge.CBP}' THEN count END) AS CBP
FROM '{EMP}' GROUP BY 1 ORDER BY 1
""")
head["date"] = pd.to_datetime(head.snapshot_ym, format="%Y%m")
fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharex=True)
for ax, col, c in zip(axes, ["ICE", "CBP"], [C_ICE, C_CBP]):
    ax.plot(head.date, head[col], color=c, marker="o", ms=3, lw=2)
    ax.set(title=f"{col} headcount", ylabel="Employees"); ax.set_ylim(bottom
    comma_y(ax); date_x(ax)
peak = head.loc[head.ICE.idxmax()]
axes[0].annotate(f"peak {int(peak.ICE):,}\n{ym_label(peak.snapshot_ym)}", (p
    textcoords="offset points", xytext=(-60, -14), fontsize=9,
fig.tight_layout()
aug25 = int(head.query("snapshot_ym=='202508'").ICE.iloc[0]); latest = int(h
print(f"ICE: {ym_label('202508')} = {aug25:,} -> peak {ym_label(peak.snapsh
    f" -> {ym_label(head.iloc[-1].snapshot_ym)} = {latest:,}")
print(f"ICE change since the Jan 2026 peak: {latest-int(peak.ICE):+,,} ({(lat

```

ICE: Aug 2025 = 20,848 -> peak Jan 2026 = 30,461 -> Jul 2026 = 28,861

ICE change since the Jan 2026 peak: -1,600 (-5.3%)



```

In [13]: pct_expand = (int(peak.ICE) / aug25 - 1) * 100
pct_slip = (latest / peak.ICE - 1) * 100
trend_word = "edged down" if pct_slip > -10 else "dropped"
display(Markdown(
    f"The surge did reach the rolls: ICE headcount jumps from about {aug
    f"of {int(peak.ICE):,} in {ym_label(peak.snapshot_ym)} - a ~{pct_exp
    f"months. It has since {trend_word} - by the latest month ICE is at
    f"down roughly {int(peak.ICE)-latest:,} ({pct_slip:.0f}%) from the peak,
    f"continued. ICE remains well above its pre-surge level, but Sections 3-
    f"themselves leaving. CBP grows slowly and steadily, with no reversal. (
    f" hires-minus-departures flow don't match exactly - a data quirk covered
    f"of the staffing story.)"
))

```

The surge **did** reach the rolls: ICE headcount jumps from about 20,848 (Aug 2025) to a peak of **30,461 in Jan 2026** — a ~46% expansion in five months. It has since **edged down** — by the latest month ICE is at about 28,861, down roughly 1,600 (-5%) from the peak, even though some hiring continued. ICE remains well above its pre-surge level, but Sections 3–4 show it is the surge hires themselves leaving. CBP grows slowly and steadily, with no reversal. (Headcount and the hires-minus-departures flow don't match exactly — a data quirk covered in caveat 4 of §5, not part of the staffing story.)

3. Who are the surge hires? (Building a profile)

With no person IDs, to ask "are the *surge hires* the ones leaving" we first need a **profile** — the job attributes that separate surge hires from normal ICE hires. We scan each attribute and rank its values by how much more common they are among surge hires than in normal hiring. We measure that with "**x vs. baseline**" = **(share among surge hires) ÷ (share among 2024 hires)**; a value of 3.0 means it is three times as common in the surge. The two decisive attributes — **job series** and **pay plan** — are shown as tables; grade, appointment type, and age are summarized in the reading that follows.

```
In [14]: BASE = "event_ym BETWEEN '202401' AND '202412'"
SURGE = f"event_ym BETWEEN '{surge.SURGE_WINDOW[0]}' AND '{surge.SURGE_WINDOW[1]}'"
WH = f"agy_code='{surge.ICE}' AND {surge.NEW_HIRE}"
Bt, St = con.execute(f"SELECT SUM(CASE WHEN {BASE} THEN count END), SUM(CASE WHEN {SURGE} THEN count END) FROM {WH}")

def fingerprint(col, min_n=150):
    d = q(f"SELECT {col} AS value, SUM(CASE WHEN {BASE} THEN count END) b, SUM(CASE WHEN {SURGE} THEN count END) s FROM {WH}")
    d[["b", "s"]] = d[["b", "s"]].fillna(0)
    d["% of surge"] = (100*d.s/St).round(1)
    d["% of 2024"] = (100*d.b/Bt).round(1)
    d["x vs 2024"] = (d["% of surge"]/d["% of 2024"]).round(1)
    d["surge hires"] = d.s.astype(int)
    return (d[d.s>=min_n].sort_values("s", ascending=False)
            .rename(columns={"value": col})
            [[col, "surge hires", "% of surge", "% of 2024", "x vs 2024"]].reset_index())

labels = {"occupational_series_code": "Job series", "pay_plan_code": "Pay plan",
          "appointment_type": "Appointment type", "grade": "Grade", "age_bracket": "Age bracket"}
display(Markdown(f"**ICE new hires – 2024 baseline = {int(Bt):,} &nbsp;·&nbsp;"))
fp_tables = {}
for c in ["occupational_series_code", "pay_plan_code"]:
    fp_df = fingerprint(c)
    fp_tables[c] = fp_df
    disp = fp_df.copy()
    if c == "occupational_series_code":
        disp[c] = disp[c].map(series_label) # show "1801 – General Inspector"
    disp = disp.rename(columns={c: labels[c]})
    display(Markdown(f"**{labels[c]}**"))
    display(show(disp, pct=["% of surge", "% of 2024"]))

# Secondary markers (appointment type, age, grade), used in the "Reading the data" section
# not shown as tables since job series and pay plan already carry the finding
```

```
fp_appt = fingerprint("appointment_type", min_n=1)
fp_age = fingerprint("age_bracket", min_n=1)
term_row = fp_appt[fp_appt.appointment_type.str.contains("NONPERMANENT", cas
age_older = fp_age[fp_age.age_bracket.isin(["50-54", "55-59"])]
gl_grades = q(f"SELECT grade, SUM(count) n FROM '{ACC}' WHERE {WH} AND {SURC
gl_top_grades = "/" .join(sorted(gl_grades.head(3).grade))
```

ICE new hires — 2024 baseline = 1,334 · surge window = 10,133

Job series

Job series	surge hires	% of surge	% of 2024	x vs 2024
1801 — General Inspection, Investigation, Enforcement, and Compliance Series	6,445	63.6%	19.9%	3.2
1811 — Criminal Investigation	2,335	23.0%	19.2%	1.2
0905 — Attorney	987	9.7%	25.3%	0.4

Pay plan

Pay plan	surge hires	% of surge	% of 2024	x vs 2024
GL	6,884	67.9%	32.6%	2.1
GS	3,244	32.0%	67.1%	0.5

```
In [15]: s_df = fp_tables["occupational_series_code"].set_index("occupational_series_
p_df = fp_tables["pay_plan_code"].set_index("pay_plan_code")
r1801, r1811, rGL = s_df.loc["1801"], s_df.loc["1811"], p_df.loc["GL"]
combined_1801_1811 = r1801["% of surge"] + r1811["% of surge"]

display(Markdown(
    f"*Reading the profile:* \n\n"
    f"- **Job series** is the sharpest marker: series **1801** (*{occ_names.
    f"immigration-enforcement / deportation officer series) is about {r1801[
    f" hires versus {r1801['% of 2024']:.0f}% in normal hiring (**{r1801['x v
    f" adding series **1811** (*{occ_names.get('1811', '')}*) covers about {cc
    f"- **Pay plan `GL`** (the entry law-enforcement officer pay plan) is ab
    f" surge hires versus {rGL['% of 2024']:.0f}% normally (**{rGL['x vs 2024
    f" **GL-{gl_top_grades}**.\n"
    f"- Two secondary markers: an unusual share of **term (non-permanent) ap
    f" (**{term_row['x vs 2024']:.1f}***), and an **older age mix** (the 50-5
    f" {age_older['x vs 2024'].min():.1f}—{age_older['x vs 2024'].max():.1f}x
    f" entry-level law-enforcement.\n\n"
    f" So we define the surge hires as **pay plan `GL` + job series 1801/1811
    f" those jobs before the surge, so any jump in departures from them is, b
    f" hires — **and this does not use length of service**, which is muddled
    f" (see §5)."
```

Reading the profile:

- **Job series** is the sharpest marker: series **1801** (*General Inspection, Investigation, Enforcement, and Compliance Series* — the ICE immigration-enforcement / deportation officer series) is about 64% of surge hires versus 20% in normal hiring (**3.2× as common**); adding series **1811** (*Criminal Investigation*) covers about 87% of the surge.
- **Pay plan GL** (the entry law-enforcement officer pay plan) is about 68% of surge hires versus 33% normally (**2.1×**), at grades **GL-05/07/09**.
- Two secondary markers: an unusual share of **term (non-permanent) appointments** (**3.0×**), and an **older age mix** (the 50–59 brackets run 1.5–2.8× normal) — atypical for entry-level law-enforcement.

So we define the surge hires as **pay plan GL + job series 1801/1811**. Almost no one at ICE held those jobs before the surge, so any jump in departures from them is, by construction, the surge hires — **and this does not use length of service**, which is muddled by prior federal experience (see §5).

4. Are the surge hires the ones leaving — and are they quitting or being let go?

Now apply the same profile to **departures**: exits from the GL-1801/1811 jobs over time.

```
In [16]: fp = surge.SURGE_FP
fp_sep_year = q(f"""
    SELECT substr(event_ym,1,4) AS year, SUM(count) AS fp_separations,
           SUM(CASE WHEN separation_category='QUIT' THEN count END) AS quits,
           SUM(CASE WHEN separation_category LIKE 'TERMINATION%' THEN count END) AS
    FROM '{SEP}' WHERE agy_code='{surge.ICE}' AND {fp} GROUP BY 1 ORDER BY 1
    """)
show(fp_sep_year.rename(columns={"year":"Year","fp_separations":"Departures",
                                "quits":"Quits","terminations":"Terminations"}))
```

Out[16]:

Year	Departures	Quits	Terminations
2018	28	19	—
2019	27	7	—
2020	46	16	1
2021	43	29	—
2022	37	19	—
2023	27	12	—
2024	32	21	—
2025	739	564	148
2026	1,303	840	393

Year	Departures	Quits	Terminations
2018	28	19	—
2019	27	7	—
2020	46	16	1
2021	43	29	—
2022	37	19	—
2023	27	12	—
2024	32	21	—
2025	739	564	148
2026	1,303	840	393

```
In [17]: pre_dep = fp_sep_year[(fp_sep_year.year >= "2018") & (fp_sep_year.year <= "2024")]
dep_2025 = int(fp_sep_year.loc[fp_sep_year.year == "2025", "fp_separations"])
dep_2026 = int(fp_sep_year.loc[fp_sep_year.year == "2026", "fp_separations"])
months_2026 = int(surge.LATEST_MONTH[4:]) # e.g. "202607" -> 7 months of 2026
pct_vs_2025 = 100 * (dep_2026 - dep_2025) / dep_2025
pre_dep_monthly, now_monthly = pre_dep.mean() / 12, dep_2026 / months_2026
pace_x = now_monthly / pre_dep_monthly

display(Markdown(
    f"Departures from these jobs were a flat {int(pre_dep.min())}–{int(pre_dep.max())} per year from 2018 through 2024. They jump to {dep_2025} in 2025 and {dep_2026} in just the first 7 months of 2026 — already {pct_vs_2025:.0f}% more than all of 2025, in {'barely half' if months_2026 <= 7 else 'well under'} the year — a monthly pace roughly {pace_x:.0f}x the pre-surge norm. The exits begin the same month the hiring does (Sep 2025). The monthly picture, next to hiring, is in the chart further down."
))
```

Departures from these jobs were a **flat 27–46 per year from 2018 through 2024**. They jump to **739 in 2025** and **1,303 in just the first 7 months of 2026** — already 76% more than all of 2025, in **barely half the year** — a monthly pace roughly **65x the pre-surge norm**. The exits begin the same month the hiring does (Sep 2025). The monthly picture, next to hiring, is in the chart further down.

Let go, or leaving on their own?

The departure category tells voluntary quits apart from involuntary terminations.

```
In [18]: why = q(f"""
SELECT separation_category, SUM(count) AS n
FROM '{SEP}' WHERE agy_code='{surge.ICE}' AND {fp} AND event_ym>='202509'
GROUP BY 1 ORDER BY n DESC
""")
why["share_%"] = (100*why.n/why.n.sum()).round(1)
q_ct = int(why.loc[why.separation_category=='QUIT', 'n'].iloc[0])
t_ct = int(why.loc[why.separation_category.str.startswith('TERMINATION'), 'n'].iloc[0])
display(show(why.rename(columns={"separation_category": "Departure type", "n": "People"})))
print(f"Voluntary QUIT: {q_ct} ({100*q_ct/why.n.sum():.0f}%) Involuntary TERMINATION: {t_ct} ({100*t_ct/why.n.sum():.0f}%)"
```

Departure type	People	share_%
QUIT	1,374	69.1%
TERMINATION (EXPIRED APPT/OTHER)	538	27.1%
TRANSFER OUT - INDIVIDUAL TRANSFER	38	1.9%
OTHER SEPARATION	34	1.7%
RETIREMENT - VOLUNTARY	3	0.2%

Voluntary QUIT: 1,374 (69%) Involuntary TERMINATION: 538 (27%)

```
In [19]: q_pct, t_pct = 100 * q_ct / why.n.sum(), 100 * t_ct / why.n.sum()
display(Markdown(
```

```

f"*Most are voluntary departures, not removals.* Of departures from the
f"2025, about **{q_pct:.0f}% are voluntary quits** and about **{t_pct:.0f}%
f"\expired appointment / other.\" The termination category combines **term
f"appointments reaching their end** (the {term_row['x vs 2024']:.1f}% term-appointment share from §3) with any
f"with any probationary removals, and the data does not separate the two
))

```

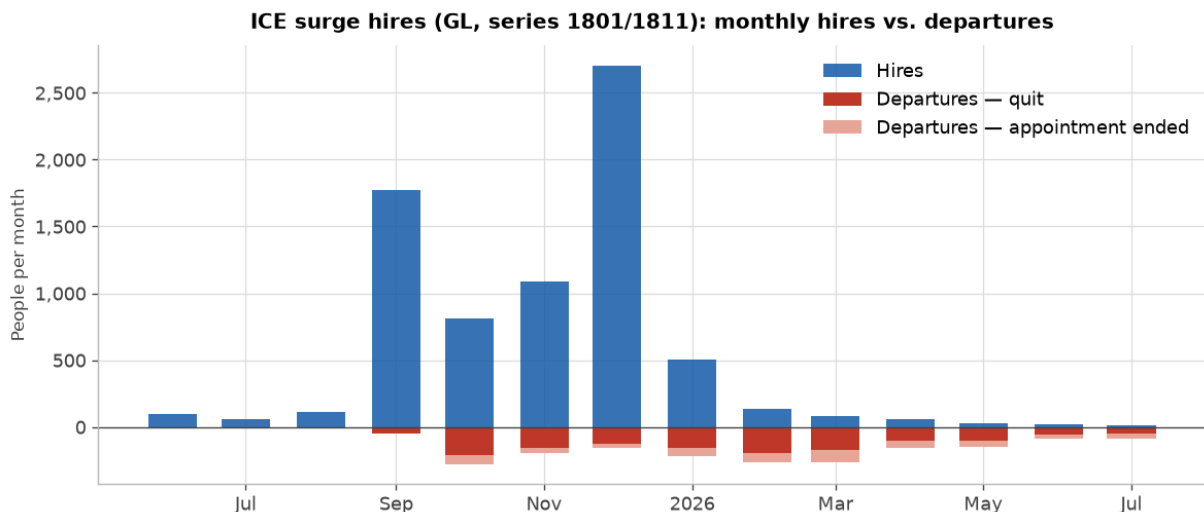
Most are voluntary departures, not removals. Of departures from the surge hires since Sep 2025, about **69% are voluntary quits** and about **27% are terminations** coded "expired appointment / other." The termination category combines **term (non-permanent) appointments reaching their end** (the 3.0× term-appointment share from §3) with any probationary removals, and the data does not separate the two.

```

In [20]: # Hires (up) vs. departures (down, split into quit vs. term-ended), on ONE series
# proportion leaving reads consistently: departures are small next to the hires
# once hiring stops.
comp = q(f"""
    WITH h AS (SELECT event_ym, SUM(count) hires FROM '{ACC}' WHERE agy_code='{surge.ICE}'
    s AS (SELECT event_ym,
        SUM(CASE WHEN separation_category='QUIT' THEN count END) quits,
        SUM(CASE WHEN separation_category LIKE 'TERMINATION%' THEN count END) terms,
        SUM(count) exits
    FROM '{SEP}' WHERE agy_code='{surge.ICE}' AND {fp} GROUP BY 1)
    SELECT COALESCE(h.event_ym,s.event_ym) event_ym, COALESCE(hires,0) hires,
        COALESCE(quits,0) quits, COALESCE(terms,0) terms, COALESCE(exits,0) exits
    FROM h FULL JOIN s USING(event_ym)
    WHERE COALESCE(h.event_ym,s.event_ym) >= '202506' ORDER BY 1
""")
comp["date"] = pd.to_datetime(comp.event_ym, format="%Y%m")
fig, ax = plt.subplots(figsize=(10, 4.4))
ax.bar(comp.date, comp.hires, width=20, color=C_ICE, alpha=.9, label="Hires")
ax.bar(comp.date, -comp.quits, width=20, color=C_EXIT, label="Departures - quits")
ax.bar(comp.date, -comp.terms, bottom=-comp.quits, width=20, color="#e8a598",
    label="Departures - appointment ended")
ax.axhline(0, color="#444", lw=.8)
ax.yaxis.set_major_formatter(mtick.FuncFormatter(lambda x, _: f"{abs(x):.0f}"))
ax.set(title="ICE surge hires (GL, series 1801/1811): monthly hires vs. departures",
    ylabel="People per month"); date_x(ax); ax.legend(loc="upper right");
comp_disp = comp.assign(Month=comp.event_ym.map(ym_label))[["Month", "hires", "quits", "terms", "exits"]]
comp_disp.columns = ["Month", "Hires", "Quits", "Appointment ended", "Departures"]
display(show(comp_disp))

```

Month	Hires	Quits	Appointment ended	Departures total
Jun 2025	98	3	0	4
Jul 2025	62	5	1	8
Aug 2025	109	3	2	8
Sep 2025	1,769	46	6	53
Oct 2025	811	210	66	276
Nov 2025	1,092	156	39	199
Dec 2025	2,704	122	34	156
Jan 2026	508	158	63	235
Feb 2026	134	192	74	270
Mar 2026	79	174	87	277
Apr 2026	57	104	56	165
May 2026	26	102	47	162
Jun 2026	22	60	26	96
Jul 2026	12	50	40	98



A wall of hiring in **Sep 2025 – Jan 2026**, then hiring falls off a cliff while departures stay elevated — so from about Feb 2026 the group **shrinks every month**. That is the mechanism behind the headcount reversal in §2.

4b. How many leave soon after being hired — and is that rate unusual?

Because these jobs were nearly empty before the surge, we can watch the **whole group's** departures pile up against its own hires — a running early-departure rate that

does **not** depend on length of service (so it also captures the ~30% of surge hires who had prior federal experience).

```
In [21]: cohort = q(f"""
    WITH h AS (SELECT event_ym, SUM(count) hires FROM '{ACC}' WHERE agy_code='
        s AS (SELECT event_ym, SUM(count) exits FROM '{SEP}' WHERE agy_code='
    SELECT m.event_ym,
        SUM(COALESCE(hires,0)) OVER w AS cum_hires,
        SUM(COALESCE(exits,0)) OVER w AS cum_exits,
        ROUND(100.0*SUM(COALESCE(exits,0)) OVER w / NULLIF(SUM(COALESCE(hires,0))
    FROM (SELECT event_ym FROM h UNION SELECT event_ym FROM s) m
    LEFT JOIN h USING(event_ym) LEFT JOIN s USING(event_ym)
    WHERE m.event_ym >= '202509' WINDOW w AS (ORDER BY m.event_ym) ORDER BY m.
    """)
    cohort["date"] = pd.to_datetime(cohort.event_ym, format="%Y%m")
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(cohort.date, cohort.pct_of_cohort_exited, marker="o", ms=5, lw=2, cc
    ax.yaxis.set_major_formatter(mtick.PercentFormatter(decimals=0))
    ax.set(title="Share of the ICE surge hires who have already left (any length
        ylabel="Departures ÷ hires, running total"); date_x(ax); fig.tight_la
    final = cohort.iloc[-1]
    print(f"By {ym_label(final.event_ym)}: {int(final.cum_exits):,} of {int(fina
        f"= {final.pct_of_cohort_exited}% – all within their first year, and s
```

By Jul 2026: 1,987 of 7,214 surge hires have left = 27.5% – all within their first year, and still climbing.



```
In [22]: display(Markdown(
    f"About **{final.pct_of_cohort_exited:.0f}%** of surge hires ({int(final
    f"{int(final.cum_hires):,}) have already left, and the share climbs ever
    f"stopped while departures continue.\n\n"
    f"Is that a lot? We can't yet answer with an ICE *first-year* rate – the
    f"Jan 2026, so by {ym_label(final.event_ym)} the earliest of them have c
    f"{{(pd.to_datetime(final.event_ym, format='%Y%m') - pd.Timestamp('2025-0
    f"the job, still short of a full year for the cohort as a whole, and anr
    f"mislead. What we *can* compute is the **completed** first-year rate fo
    f"*CBP's entry law-enforcement officers* – Border Patrol Agents (serie
    f"Enforcement*, GL) and CBP Officers (series 1895, *Customs and Border F
    f"enforcement mission, same entry grades, same academy onboarding, but h
    f"their cohorts have each had a full year or more to play out. Those giv
```

```

    f"to hold ICE's partial-year loss against — a comparison of *horizons*,
    f"CBP measure: departures with under a year of service ÷ new-entrant hir
))

```

About **28%** of surge hires (1,987 of 7,214) have already left, and the share climbs every month because hiring stopped while departures continue.

Is that a lot? We can't yet answer with an ICE *first-year* rate — these hires arrived Sep 2025 – Jan 2026, so by Jul 2026 the earliest of them have only about 10 months on the job, still short of a full year for the cohort as a whole, and annualizing a partial year would mislead. What we *can* compute is the **completed** first-year rate for a genuinely similar group: **CBP's entry law-enforcement officers** — Border Patrol Agents (series 1896, *Border Patrol Enforcement*, GL) and CBP Officers (series 1895, *Customs and Border Protection*). Same DHS enforcement mission, same entry grades, same academy onboarding, but hired in volume every year, so their cohorts have each had a full year or more to play out. Those give honest full-year benchmarks to hold ICE's partial-year loss against — a comparison of *horizons*, not rate-against-rate. (The CBP measure: departures with under a year of service ÷ new-entrant hires.)

```

In [23]: def first_year_rate(agy, fp_, y0, y1):
    d = q(f"""
        WITH h AS (SELECT SUM(CASE WHEN {surge.dbl('length_of_service_years')}
            FROM '{ACC}' WHERE agy_code='{agy}' AND {surge.NEW_HIRE} AN
            AND substr(event_ym,1,4) BETWEEN '{y0}' AND '{y1}'),
        s AS (SELECT SUM(CASE WHEN {surge.dbl('length_of_service_years')}
            FROM '{SEP}' WHERE agy_code='{agy}' AND {fp_}
            AND substr(event_ym,1,4) BETWEEN '{y0}' AND '{y1}')
        SELECT hires, exits_lt1yr, ROUND(100.0*exits_lt1yr/NULLIF(hires,0),1)
    """).iloc[0]
    return int(d.hires), int(d.exits_lt1yr), d.first_yr_sep_pct

# No ICE row on purpose: these hires are only a few months into their first
# rate isn't computable yet. ICE's observed loss so far (chart above) is con
# CBP benchmarks by horizon, in words, below.
rows = [
    ("CBP Border Patrol (1896 GL) – 2023–24 classes", "HSBD", "pay_plan_code='
    ("CBP Border Patrol (1896 GL) – 2019–21 rapid-hire", "HSBD", "pay_plan_coc
    ("CBP Officer (1895) – 2023–24 classes", "HSBD", "occupational_series_code
]
cbp = pd.DataFrame([(lbl,)+first_year_rate(a,f,y0,y1) for lbl,a,f,y0,y1 in r
                    columns=["group","new_entrant_hires","exits_under_1yr","f
display(Markdown("**Reference – CBP departures within the _full_ first year
display(show(cbp.rename(columns={"group":"CBP entry law-enforcement group",
                    "exits_under_1yr":"Left within first year",
                    pct=["First-year rate"])))
print(f"ICE surge hires so far (chart above): {int(final.cum_exits):,} of {i
      f"= {final.pct_of_cohort_exited}% – before their first year is complet

```

Reference — CBP departures within the *full* first year of service (completed cohorts)

CBP entry law-enforcement group	New-entrant hires	Left within first year	First-year rate
CBP Border Patrol (1896 GL) — 2023-24 classes	1,977	226	11.4%
CBP Border Patrol (1896 GL) — 2019-21 rapid-hire	3,275	944	28.8%
CBP Officer (1895) — 2023-24 classes	2,075	218	10.5%

ICE surge hires so far (chart above): 1,987 of 7,214 left = 27.5% — before their first year is complete, so not shown as a rate.

```
In [24]: cbp_normal = cbp[cbp.group.str.contains("2023-24")]["first_yr_sep_%"]
cbp_surge = cbp[cbp.group.str.contains("2019-21")]["first_yr_sep_%"].iloc[0]
position = "near the high end of" if final.pct_of_cohort_exited <= cbp_surge
display(Markdown(
    f"*The verdict.* We can't state an ICE first-year rate — the cohort's
    f"(these hires are only a few months in), and annualizing a partial year
    f"say: ICE has already lost about {final.pct_of_cohort_exited:.0f}%
    f"For a full-year reference, CBP's comparable entry classes lose about *
    f"{cbp_normal.max():.0f}% over a whole year** in normal times, and even
    f"cohort — Border Patrol in 2019–21 — lost about {cbp_surge:.0f}%
    f"well under a year ICE has already passed CBP's steady-state full-year
    f"own worst rapid-hire surge — {position} that range, with its first year
    ))
```

The verdict. We can't state an ICE *first-year* rate — the cohort's first year isn't over (these hires are only a few months in), and annualizing a partial year would mislead. What we can say: ICE has **already lost about 28%** before that first year is up. For a full-year reference, CBP's comparable entry classes lose about **10–11% over a whole year** in normal times, and even CBP's roughest rapid-hire cohort — Border Patrol in **2019–21** — lost about **29% over a whole year**. So in well under a year ICE has already passed CBP's steady-state full-year loss and is closing on CBP's own worst rapid-hire surge — near the high end of that range, with its first year not yet complete.

```
In [25]: pre_surge_profile_hc = int(q(f"SELECT SUM(count) FROM '{EMP}' WHERE agy_code=
attorneys = int(q(f"SELECT SUM(count) FROM '{ACC}' WHERE agy_code='{surge.
    AND event_ym BETWEEN '{surge.SURGE_WINDOW[0]}' AND '{surge.SURGE_WINDOW[1]}'
tot_dep_window = int(q(f"SELECT SUM(count) FROM '{SEP}' WHERE agy_code='{surge.
drp_cohort = q(f"SELECT COALESCE(drp_indicator, '(blank)') AS d, SUM(count) AS c
drp_all = q(f"SELECT COALESCE(drp_indicator, '(blank)') AS d, SUM(count) AS c
drp_yes_cohort = int(drp_cohort.loc[drp_cohort.d == "Y", "n"].sum())
drp_yes_all = int(drp_all.loc[drp_all.d == "Y", "n"].sum())
window_tot = int(q(f"SELECT SUM(count) FROM '{ACC}' WHERE agy_code='{surge.ICE}'
window_fp = int(q(f"SELECT SUM(count) FROM '{ACC}' WHERE agy_code='{surge.ICE}'
fp_coverage_pct = 100 * window_fp / window_tot
prior_2025 = q(f"SELECT
    SELECT CASE WHEN {surge.dbl('length_of_service_years')} >= 3 THEN 1 ELSE 0
    FROM '{ACC}' WHERE agy_code='{surge.ICE}' AND {surge.NEW_HIRE} AND substr(
    )
prior_3plus_pct = 100 * prior_2025.set_index("is_3plus").loc[1, "n"] / prior
```

```

display(Markdown(
  f"## 5. Net effect, and what the data can and cannot say\n\n"
  f"*Net effect.* The 2025 surge was overwhelmingly ICE, reached the pay
  f"{pct_expand:.0f}%), and is now unwinding as the surge hires themselves
  f"{abs(pct_slip):.0f}% off the January peak, {final.pct_of_cohort_exited
  f"before its first year is even up, mostly by voluntary quit. The charts
  f"the caveats below cover what this data can't settle.\n\n"
  f"*Caveats (each checked or bounded in §0):*\n\n"
  f"1. *Counts, not people.* We match *departing* jobs to *hiring* jobs,
  f"tight only because ICE had few people in these entry jobs before the s
  f"(~{pre_surge_profile_hc:,} in Aug 2025, small next to the {int(final.c
  f"them).\n"
  f"2. *We track the ~{fp_coverage_pct:.0f}% we can.* The profile (GL, 1
  f"{fp_coverage_pct:.0f}% of surge-window hires – the entry officers. The
  f"~{attorneys:,} attorneys) sit in jobs with large standing populations,
  f"isolated the same way.\n"
  f"3. *Length of service ≠ time in this job.* About {prior_3plus_pct:.0
  f"arrived with 3+ years of prior federal service (veterans and rehires,
  f"0.4%, §0) – exactly why we profile by job and grade instead (full brea
  f"4. *Headcount and flow don't reconcile* (§2) – flow lags the headcou
  f"headcount for levels and departures for exits, never chained.\n"
  f"5. *Recent months are provisional* (§0 check 3) – if anything, this
  f"exits.\n"
  f"6. *Term vs. cause.* \"Termination\" mixes term-appointment expirati
  f"removals; can't be split further.\n"
  f"7. *Redaction preserves totals* (§0 check 4) – masked fields are rel
  f"8. *Not a buyout artifact.* {drp_yes_cohort} of the {int(final.cum_e
  f"and {drp_yes_all} of all {tot_dep_window:,} ICE departures in this wir
  f"deferred-resignation-program exits."
))

```

5. Net effect, and what the data can and cannot say

Net effect. The 2025 surge was overwhelmingly ICE, reached the payroll (headcount +46%), and is now unwinding as the surge hires themselves leave — headcount down 5% off the January peak, 28% of the cohort gone before its first year is even up, mostly by voluntary quit. The charts above carry the numbers; the caveats below cover what this data can't settle.

Caveats (each checked or bounded in §0):

1. **Counts, not people.** We match *departing* jobs to *hiring* jobs, not tracked individuals — tight only because ICE had few people in these entry jobs before the surge (~1,511 in Aug 2025, small next to the 7,214 hired into them).
2. **We track the ~68% we can.** The profile (GL, 1801/1811) covers about 68% of surge-window hires — the entry officers. The rest (GS officers and ~987 attorneys) sit in jobs with large standing populations, so their attrition can't be isolated the same way.
3. **Length of service ≠ time in this job.** About 30% of 2025 ICE new hires arrived with 3+ years of prior federal service (veterans and rehires, not transfers — those are 0.4%, §0) — exactly why we profile by job and grade instead (full breakdown below).
4. **Headcount and flow don't reconcile** (§2) — flow lags the headcount change, so we use headcount for levels and departures for exits, never chained.
5. **Recent months are provisional** (§0 check 3) — if anything, this **understates** recent exits.
6. **Term vs. cause.** "Termination" mixes term-appointment expirations with any for-cause removals; can't be split further.
7. **Redaction preserves totals** (§0 check 4) — masked fields are relabeled, not dropped.
8. **Not a buyout artifact.** 0 of the 1,987 cohort departures — and 0 of all 4,098 ICE departures in this window — are flagged as deferred-resignation-program exits.

Supporting detail for caveat 8 — the same DRP check, broken out by flag:

```
In [26]: drp = q(f"""
        SELECT COALESCE(drp_indicator, '(blank)') AS drp_indicator, SUM(count) AS c
        FROM '{SEP}' WHERE agy_code='{surge.ICE}' AND {fp} AND event_ym>='202509'
        GROUP BY 1 ORDER BY departures DESC
        """)
        show(drp.rename(columns={"drp_indicator": "DRP flag (Y = deferred-resignation
                                "departures": "Cohort departures"}))
```

Out [26]: **DRP flag (Y = deferred-resignation program) Cohort departures**

N	1,987
---	-------

The stock-vs-flow gap behind caveat 4, month by month. ICE headcount change vs. hires – departures around the surge: they tell the same story but don't match exactly,

most visibly at the Dec 2025 / Jan 2026 boundary, where December-effective hires only land on the January headcount.

```
In [27]: recon = q(f"""
    WITH e AS (SELECT snapshot_ym ym, SUM(count) hc FROM '{EMP}' WHERE agy_coc
    a AS (SELECT event_ym ym, SUM(count) acc FROM '{ACC}' WHERE agy_code=
    s AS (SELECT event_ym ym, SUM(count) sep FROM '{SEP}' WHERE agy_code=
    SELECT e.ym, hc, hc - LAG(hc) OVER (ORDER BY e.ym) AS d_headcount,
    acc, sep, acc - sep AS net_flow
    FROM e LEFT JOIN a ON a.ym=e.ym LEFT JOIN s ON s.ym=e.ym
    WHERE e.ym BETWEEN '202508' AND '{surge.LATEST_MONTH}' ORDER BY e.ym
    """)
recon_disp = recon.assign(Month=recon.ym.map(ym_label))[["Month","hc","d_headcount","acc","sep","net_flow"]]
recon_disp.columns = ["Month","Headcount","Change vs. prior month","Hires","Departures","Hires - departures"]
show(recon_disp)
```

Out[27]:

Month	Headcount	Change vs. prior month	Hires	Departures	Hires – departures
Aug 2025	20,848	—	215	125	90
Sep 2025	22,799	1,951	2,839	183	2,656
Oct 2025	24,593	1,794	1,598	447	1,151
Nov 2025	26,827	2,234	1,756	336	1,420
Dec 2025	28,272	1,445	3,269	497	2,772
Jan 2026	30,461	2,189	715	496	219
Feb 2026	30,355	-106	230	482	-252
Mar 2026	30,102	-253	177	453	-276
Apr 2026	29,714	-388	128	315	-187
May 2026	29,219	-495	166	359	-193
Jun 2026	29,065	-154	91	251	-160
Jul 2026	28,861	-204	78	279	-201

```
In [28]: prior = q(f"""
    SELECT CASE WHEN {surge.dbl('length_of_service_years')} < 0.15 THEN 'Truly
    WHEN {surge.dbl('length_of_service_years')} < 3 THEN 'Some prior
    ELSE '3+ years of prior federal service' END AS prior_experience
    SUM(count) AS n
    FROM '{ACC}' WHERE agy_code='{surge.ICE}' AND {surge.NEW_HIRE} AND substr(
    GROUP BY 1 ORDER BY n DESC
    """)
prior["share_%"] = (100*prior.n/prior.n.sum()).round(1)
show(prior.rename(columns={"prior_experience":"Prior federal service","n":"F"}))
```

Out [28]:

	Prior federal service	People	share_%
Truly new (under 2 months of service)		6,752	64.5%
3+ years of prior federal service		3,132	29.9%
Some prior federal service (under 3 yr)		590	5.6%

```
In [29]: p3plus_share = prior.set_index("prior_experience").loc["3+ years of prior fe
display(Markdown(
    f"About {p3plus_share:.0f}% of 2025 ICE \"new hires\" arrived with **3+
    f"service** – rehires, conversions, and moves from other agencies enteri
    f"is exactly why the surge hires have to be identified by **the job they
    f"how long they've been on a federal payroll.\n\n"
    f"---\n"
    f"*Source: OPM/EHRI hires, departures, and headcount via `impactproject/
    f"by `src/extract.py`; the surge-hire definition and windows are in `src
    f"HSBD. Data through {ym_label(surge.LATEST_MONTH)}; the most recent 1–2
))
```

About 30% of 2025 ICE "new hires" arrived with **3+ years of prior federal service** — rehires, conversions, and moves from other agencies entering as new appointments. This is exactly why the surge hires have to be identified by **the job they were hired into**, not by how long they've been on a federal payroll.

Source: OPM/EHRI hires, departures, and headcount via `impactproject/opm-ehri-data`; extract built by `src/extract.py`; the surge-hire definition and windows are in `src/surge.py`. ICE = HSBB, CBP = HSBD. Data through Jul 2026; the most recent 1–2 months are provisional.